

Hardware Technology

Di WANG

SOKENDAI / Accelerator Control Systems

2024/11/21

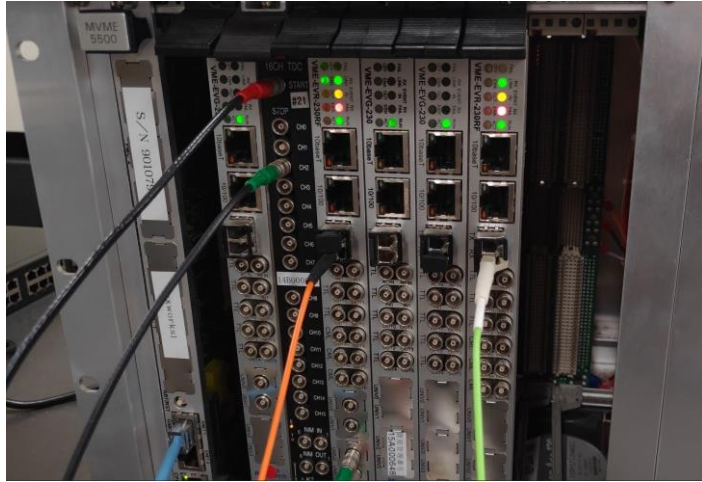
➤ **Hardware technologies**

- **Modular crates**
- **PLC**
- **Embedded control systems**

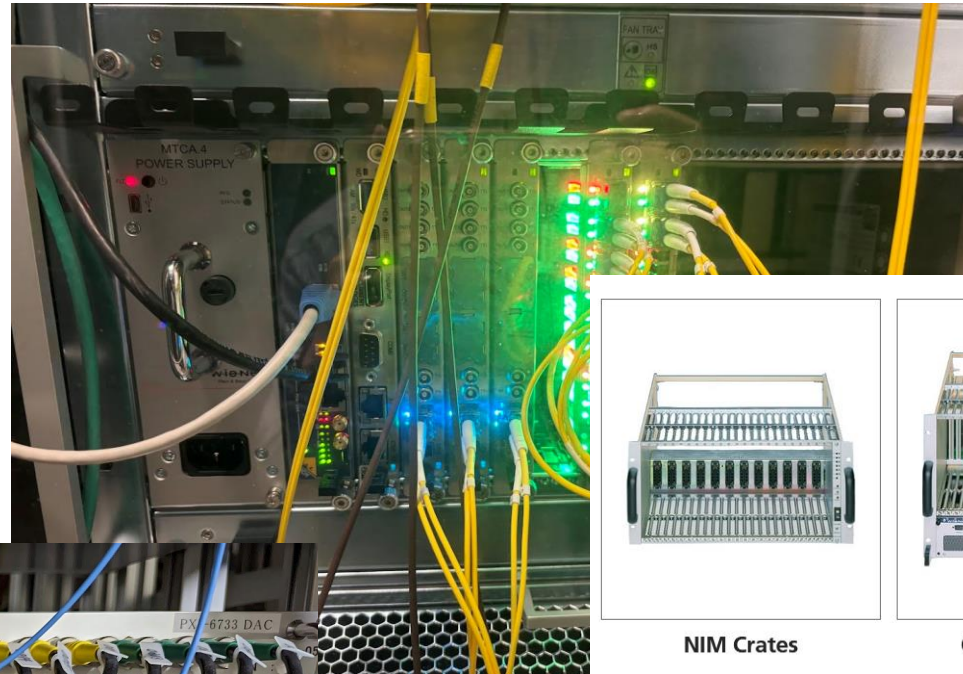
➤ **The future**

- **Citius-Altius-Fortius**

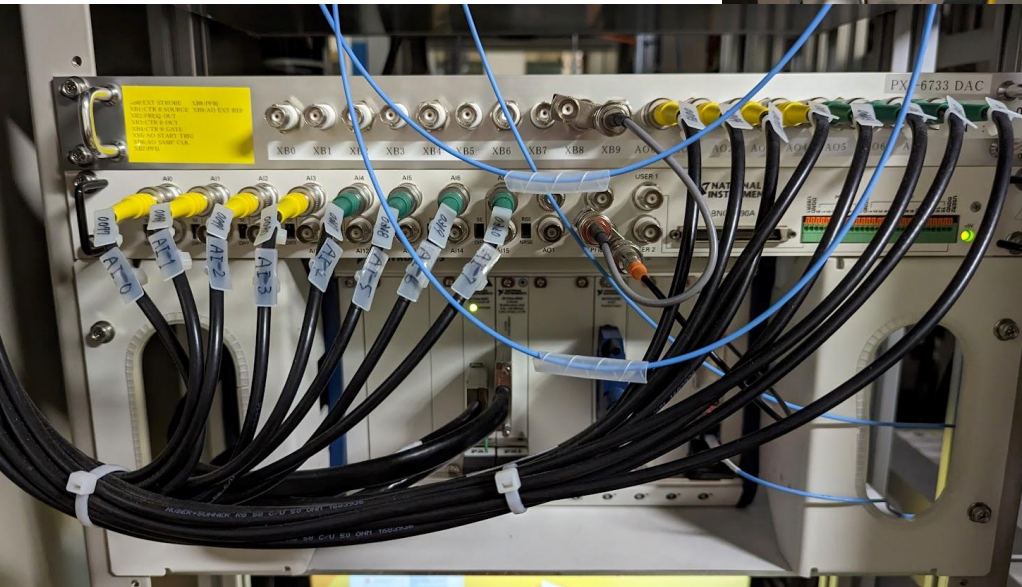
Modular crates



VME



MicroTCA



PXIe



NIM Crates



CAMAC Crates



VME Crates



VME64-x Crates



VME430 Crates



VXI & VXI-4 Crates



VXS Crates

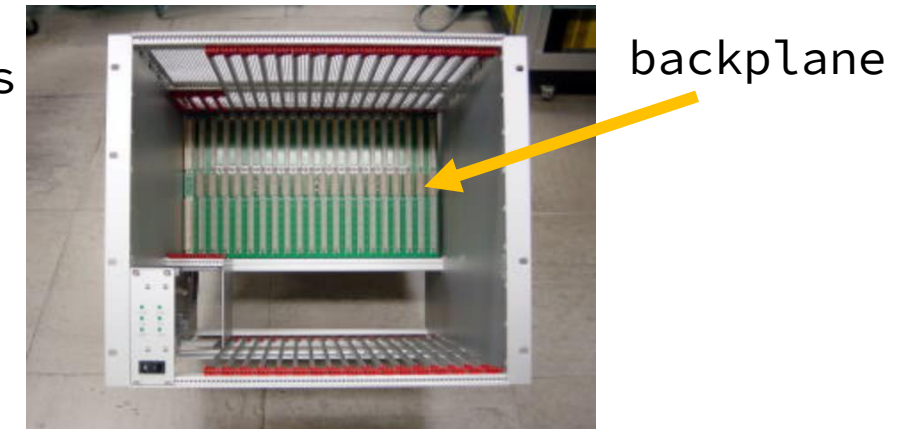
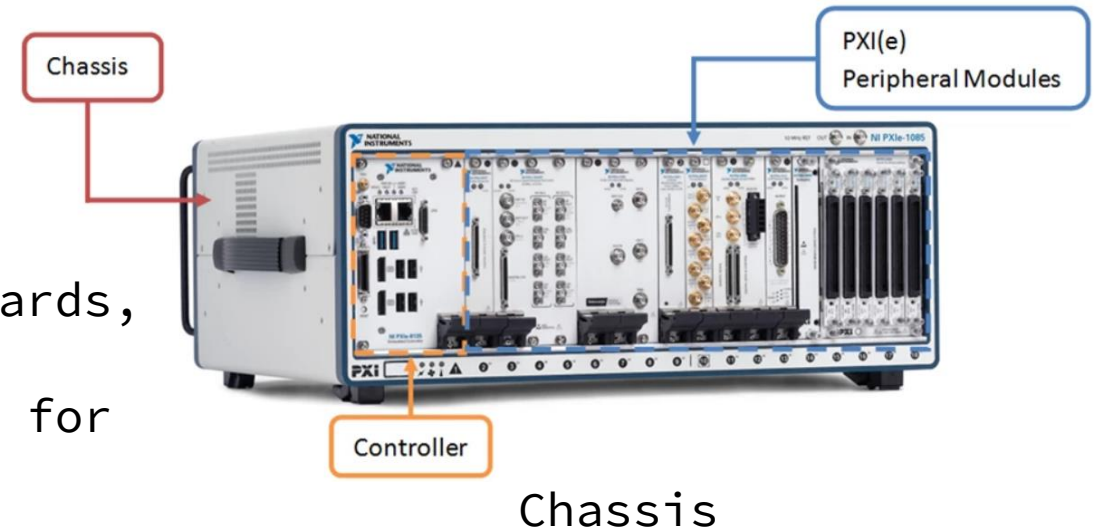


cPCI / PXI / PXIe Crates

More crates

Modular crates

- What is modular crates?
 - No strict definition
 - Modularity
 - Standardized mechanical size for chassis/module
 - Chassis/module may have several standards, e.g., 3 U or 6 U, 3 HP or 6 HP
 - Search Rack Unit and Horizontal pitch for detail
 - Use backplane to transfer signal
 - Interchangeable modules
 - Scalability
 - One can install a single module in a 12 slots crate or use all slots
 - High performance and high reliability
 - Modular crates are designed for performance and reliability-critical applications
- Synonym
 - Crate, chassis, subrack, card cage



Modular crates

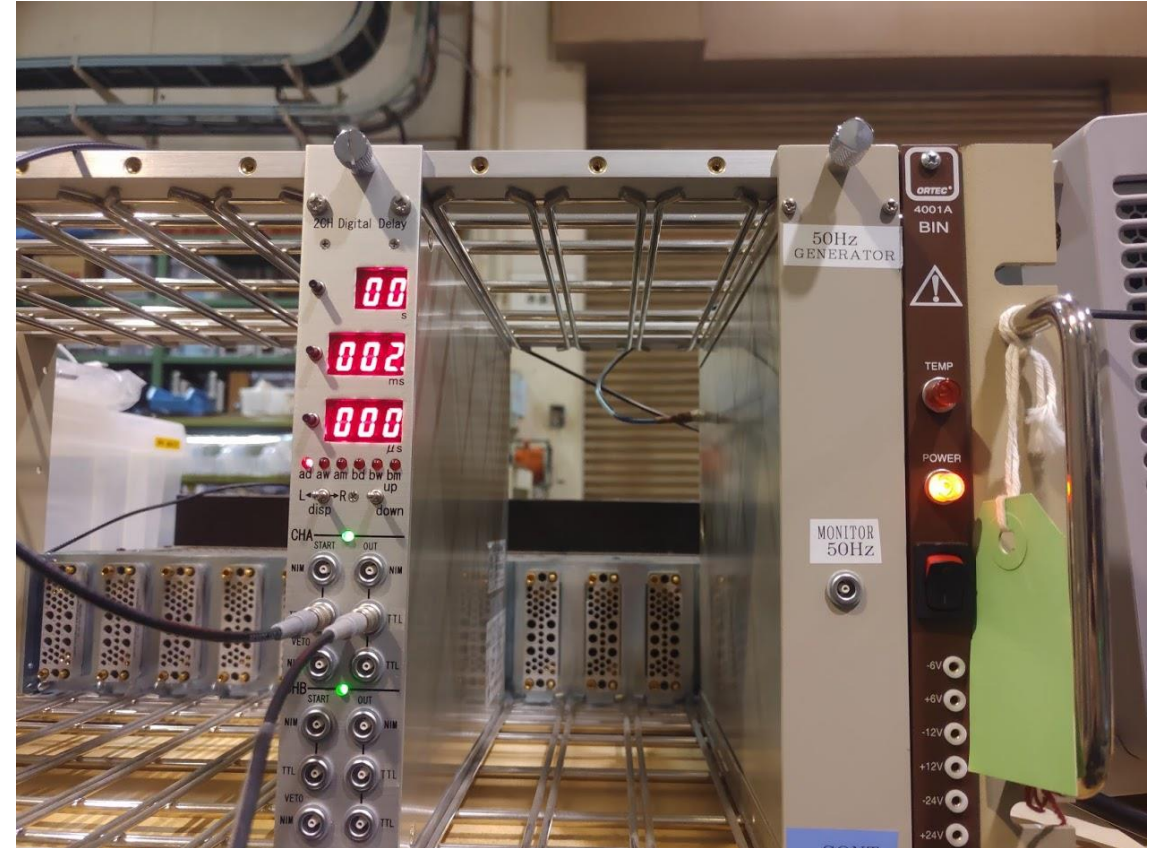
- Philosophy of modular modules
 - “Make each module do one thing well”
- Re-use generic modules in different applications
 - CPU module can be used in timing/BPM/LLRF
- 3rd party support
 - Products, services, repair
 - Competition usually gives better prices
- Upgrades are not difficult
 - 12-bit ADC -> 16-bit ADC
- But standards does not mean perfect
 - If the standard is
 - Too old
 - Bad performance, fewer suppliers, expensive
 - Too new
 - Compatibility issue, vague long-term support, few suppliers
- Unix philosophy:
 - “Small is beautiful”
 - “Make each program do one thing well”, ..

From Yamada San's slide

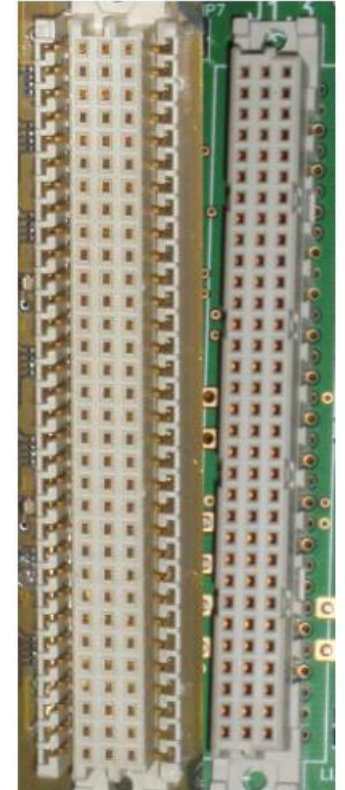
Example of modular crates

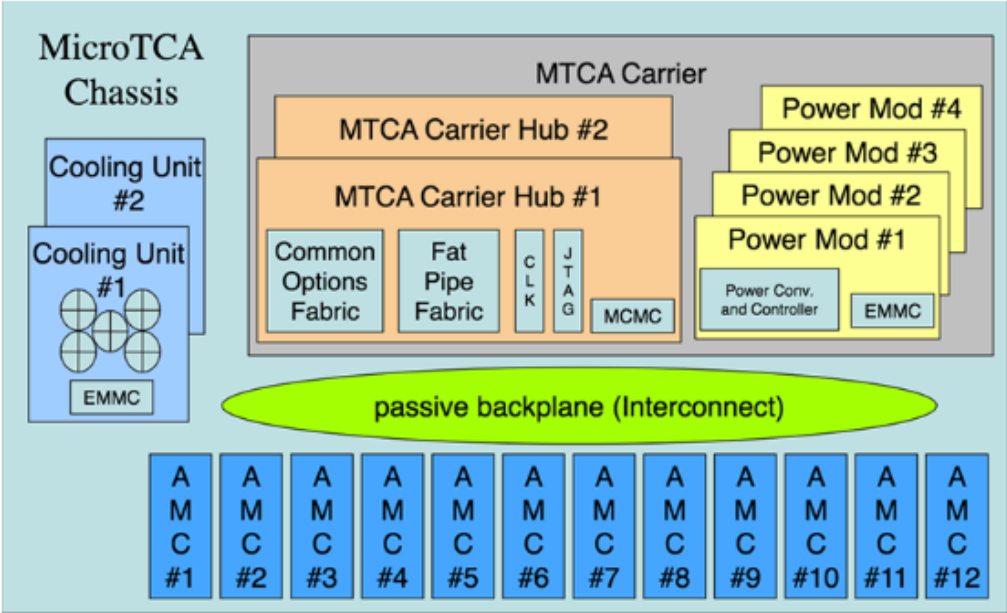
- Example of modular crates widely used in accelerator control systems
 - **NIM**
 - CAMAC
 - **VME**
 - PXI / PXIe
 - cPCI
 - ATCA / **MicroTCA**
- Of course, there are many other crate types, many are used for telecom, military defense, aerospace applications.
 - VPX
 - OpenVPX
 - PXI Multi-chassis Systems
 - AXIe

- NIM = Nuclear Instrument Modules
 - Wildly used outside of "nuclear science"
- Initially in 1960s
- No module communication through backplane
- NIM modules
 - No software (Usually)
 - Not connected to a computer (Usually)
 - Mostly used for logic signal
 - Amplifiers
 - Discriminators
 - Level converters
 - Pulse generators
 - Delay generators

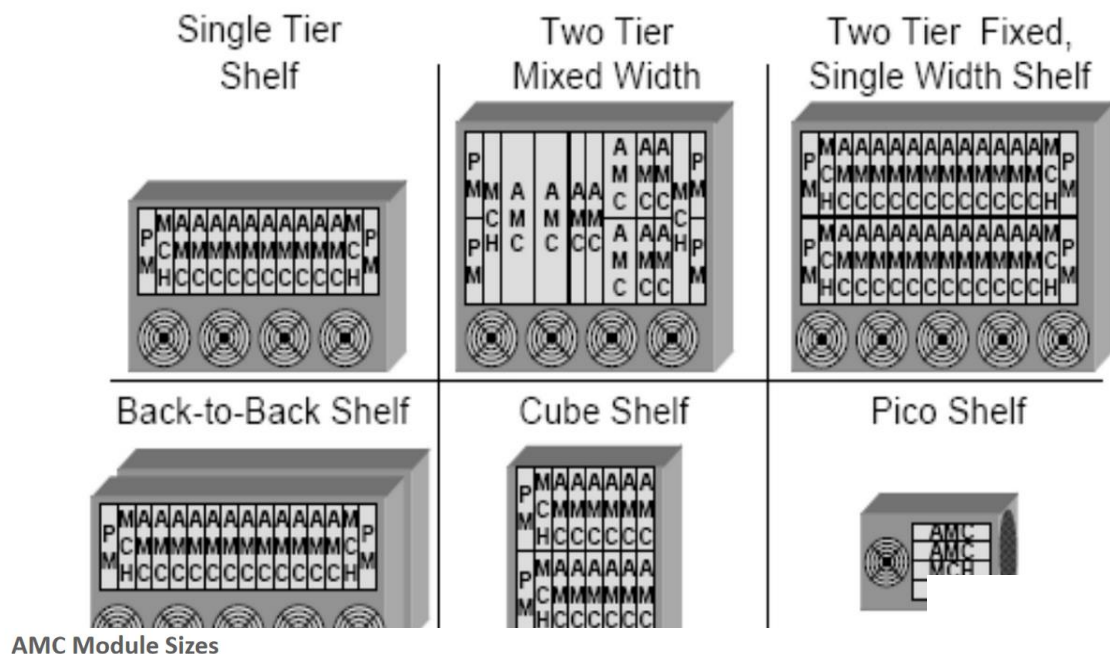


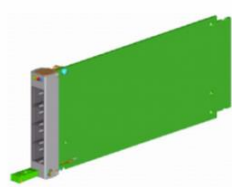
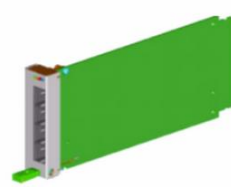
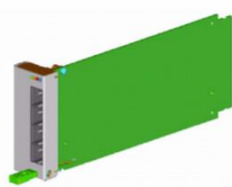
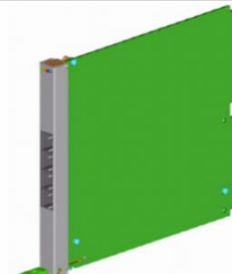
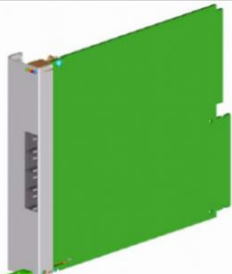
- VERSA module Eurocard
- Originated in the 1980s
- An “ecosystem” of third-party products (crates, processors, I/O modules, etc.)
- Connects modules using standardized pins and buses
- Wildly used for HEP
 - Computing, DAQ
- Provides power to all installed modules
- Specialized cards (e.g., CPU, I/O, storage) inserted into slots
- Card size
 - Heights: 3U, 6U, 9U, 1U=1.75 inch
- CPU board works as master (initiator of a transfer) and interrupt handler in the crate
- Backplane
 - Two connectors: P1 and P2
 - VME32 (right)
 - 3 rows of 32 pins each, $32 \times 3 \times 2 = 192$
 - VME64 (left)
 - 5 rows of 32 pins each, $32 \times 5 \times 2 = 320$

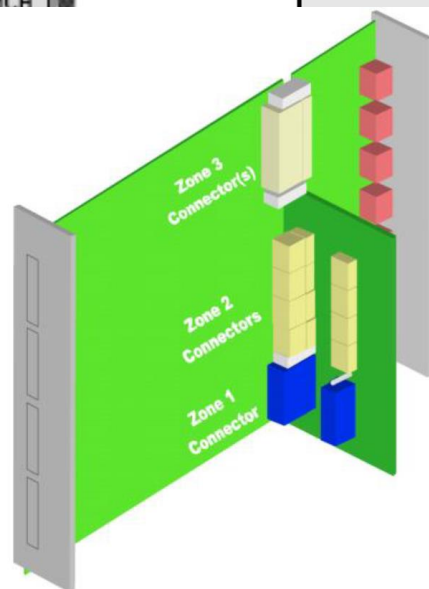
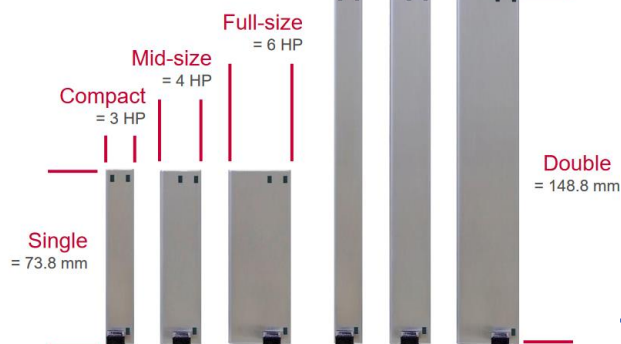
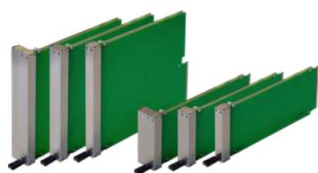
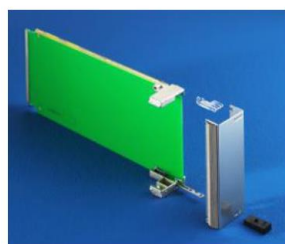


- Micro Telecommunications Computing Architecture
 - Developed and standardized by PICMG (PCI Industrial Computer Manufacturers Group)
 - Redundancy design
 - MCH, PS, cooling
 - System management based on IPMI / I2C
 - Hot-swap support for PS & cooling
 - JTAG support
 - Rear transition module support (MTCA.4)
- 

The diagram illustrates the MicroTCA Chassis architecture. It features a central 'passive backplane (Interconnect)' represented by a green oval. Above this backplane is the 'MTCA Carrier' section, which contains two 'MTCA Carrier Hub' units (Hub #1 and Hub #2). Hub #1 is connected to a 'Common Options Fabric', 'Fat Pipe Fabric', 'CLK', 'JTAG', and 'MCMC'. Hub #2 is connected to a 'Power Mod #4', 'Power Mod #3', 'Power Mod #2', and 'Power Mod #1'. The 'Power Mod #1' is further connected to a 'Power Conv. and Controller' and an 'EMMC'. To the left of the backplane are two 'Cooling Unit' blocks (Unit #1 and Unit #2), each containing an 'EMMC'. Below the backplane is a row of 12 'Rear Transition Modules' (RTMs), labeled #1 through #12, each containing an 'AMC' (Advanced Mezzanine Card).
- Glossary
 - AMC: Advanced Mezzanine Card
 - MCH MicroTCA Carrier Hub
 - CU: Cooling Unit
 - PM: Power Module
 - MCMC: MicroTCA Carrier Management Controllers
 - MMC: Module Management Controller
 - EMMC: Enhanced Module Management Controller
 - IPMI: Intelligent Platform Management Interface
 - RPM: Rear Power Module
 - RTM: Rear Transition Module
 - MCH-RTM: Extended Micro Rear Transition Module connected to MCH with Zone-3 connector



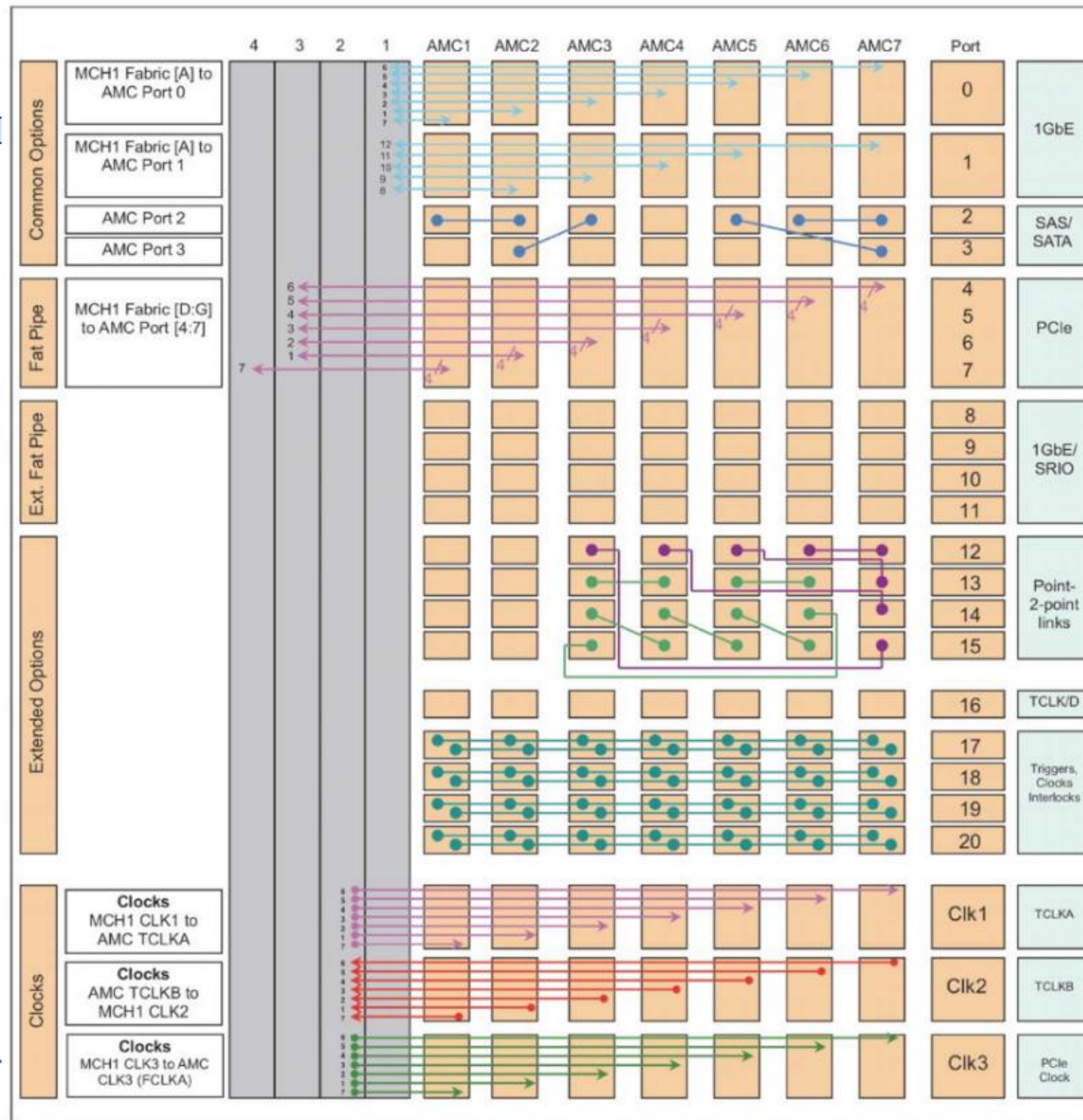
	Compact-Size (3HP)	Mid-Size (4HP)	Full-Size (6HP)
Single modules	 73.8x13.88x181.5mm	 73.8x18.96x181.5mm	 73.8x28.95x181.5mm
Double modules	 148.8x13.88x181.5mm	 148.8x18.96x181.5mm	 148.8x28.95x181.5mm



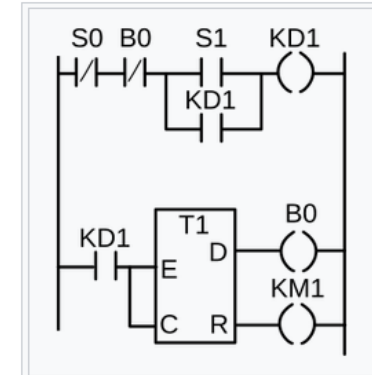
- Zone 1: Management and Power
- Zone 2: Base Interface and Fabric Interface
- Zone 3: Interface to RTM

- Fabric/fat pipes
- AMC.0
 - Base specification, management, power, clock
- AMC.1
 - PCIe, gen 1,2,3
- AMC.2
 - Ethernet, XAUI
- AMC.3
 - SATA, SAS
- AMC.4
 - Serial RapidIO

SCHROFF
crate



- Programmable Logic Controllers (PLC)
 - CPU
 - Memory
 - Power module
 - IO module
 - Communication interface
- Mainly for automation, motion control
 - Elevator
 - Accelerator safety system
- Optimized for deterministic, real-time operations
- Scan cycle
 - Read inputs
 - Execute the program
 - Write outputs
- Often use proprietary protocols
- Expansion is usually constrained to adding more IO modules
- Programming
 - Using graphical programming language called Ladder logic
 - intended to be used by people without a programming background
 - IEC 61131 standard



Example of a ladder diagram logic

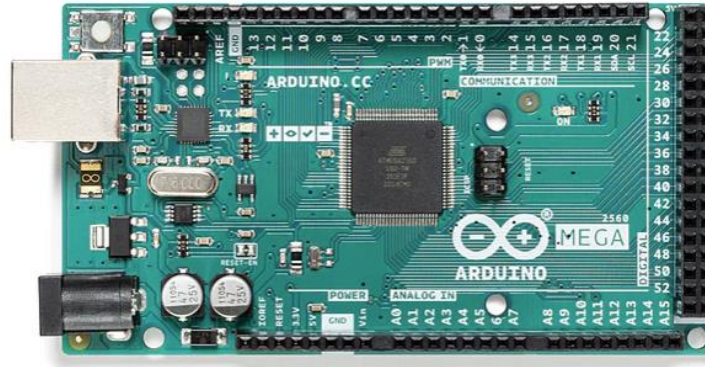
https://en.wikipedia.org/wiki/Programmable_logic_controller

Embedded control system

- Why we need embedded system for accelerator control?
 - For safety system, real-time process is critical -> PLC
 - For timing/DAQ/LLRF, performance and reliability is critical -> Modular crates
 - How about these slow DAQ systems?
 - Monitoring temperature in gallery
 - Water leak alert in tunnel
 - Checking radiation level in arc section
 - Embedded systems are widely used because of their economical cost
- Based on OS type
 - Bare metal (No OS)
 - Software simply has a loop to finish predefined tasks. May be triggered by interrupts
 - Not EPICS possible
 - Embedded OS
 - FreeRTOS, Zephyr
 - Running whole EPICS is difficult, maybe only CA (Channel Access) server?
 - Linux-based
 - OpenEmbedded, BuildRoot, Yocto
 - Full EPICS functionality

Example of embedded system

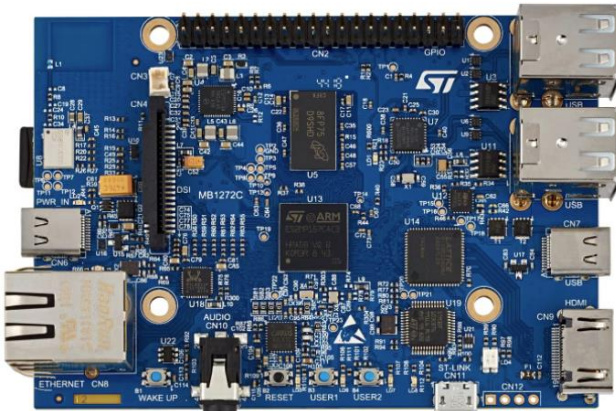
- Example of embedded system
 - MCU (microcontroller) based
 - Arduino
 - Raspberry Pi Zero
 - ESP32
 - STM32
 - CPU/MPU (microprocessor) based
 - Raspberry Pi
 - BeagleBone black
 - Orange/Banana/Lichee/Rock Pi
- Mostly are **ARM** based



<https://www.arduino.cc/>



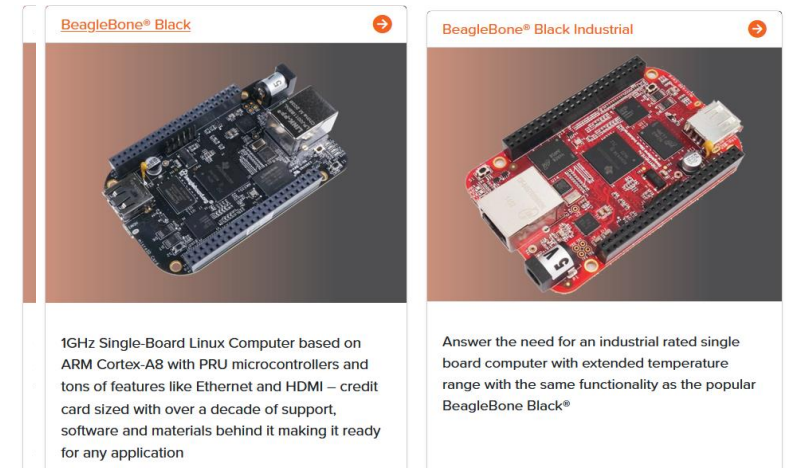
<https://www.raspberrypi.com/products/>



<https://www.st.com/en/evaluation-tools/stm32mp157d-dk1.html>

ARM: an ISA (Instruction set architecture) type

x86, ARM, RISC-V, PowerPC ...



<https://www.beagleboard.org/boards>

Typical Embedded control system

ARM board (Linux)

Raspberry Pi
(Broadcom)
BeagleBone Black (TI)
Armadillo (NXP i.MX)
Orange Pi (Rockchip)
...

Adapter board (Optional)

SPI
I²C
1-Wire
USB
Ethernet

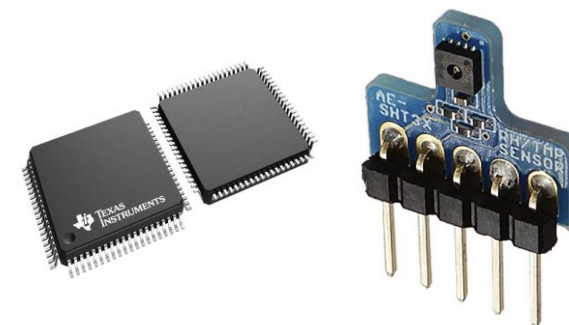
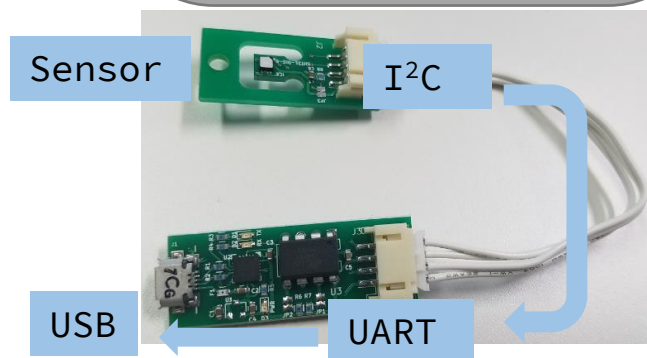
UART to USB
I²C to UART
I²C to Ethernet
SPI to UART
1-Wire to UART
...

Sensors

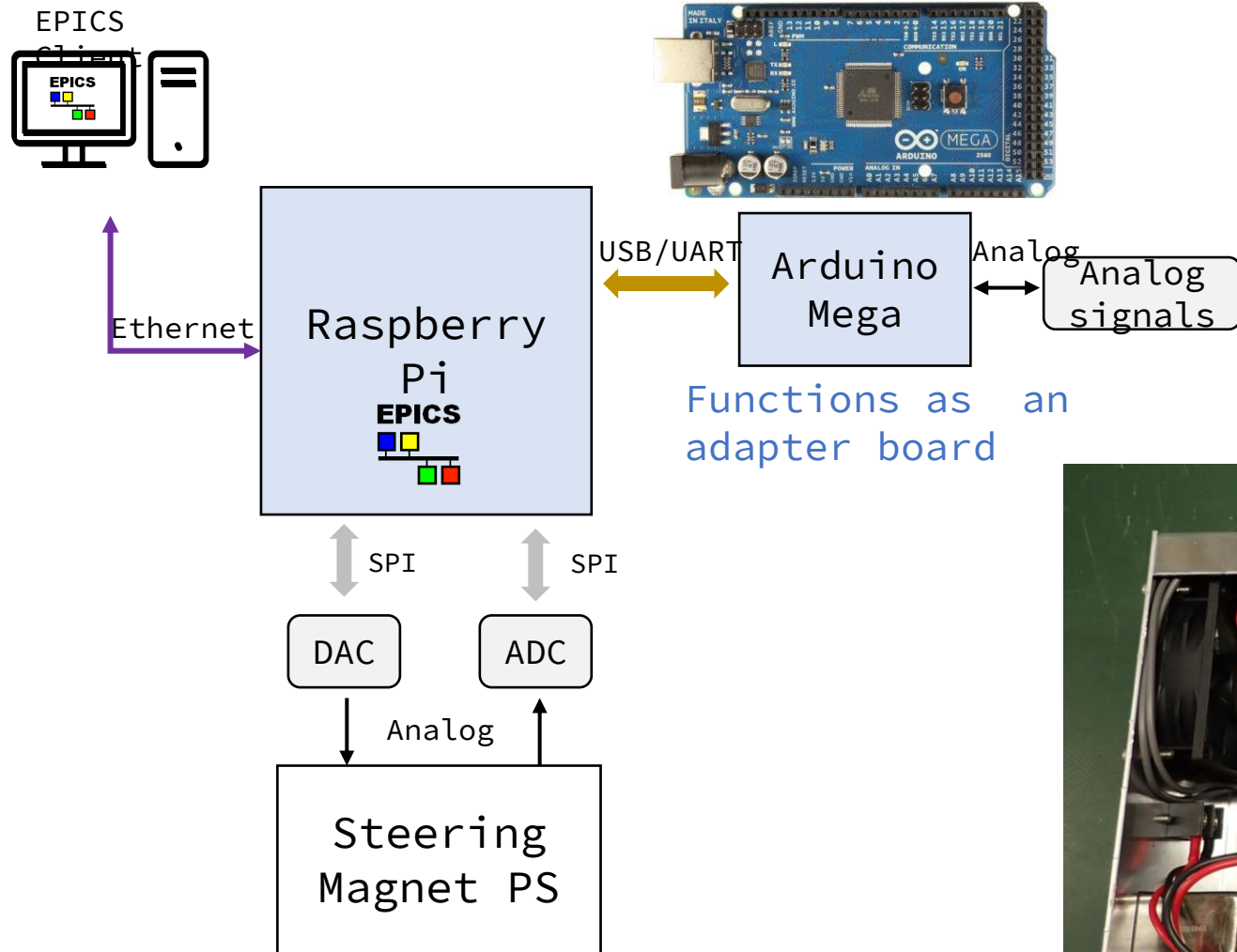
ADC & DAC
Temperature
Humidity
Radiation
IR
Smoke & Gas
Water flow
Pressure
Magnetic (Hall effect)
Tilt
Power & Energy

protocols

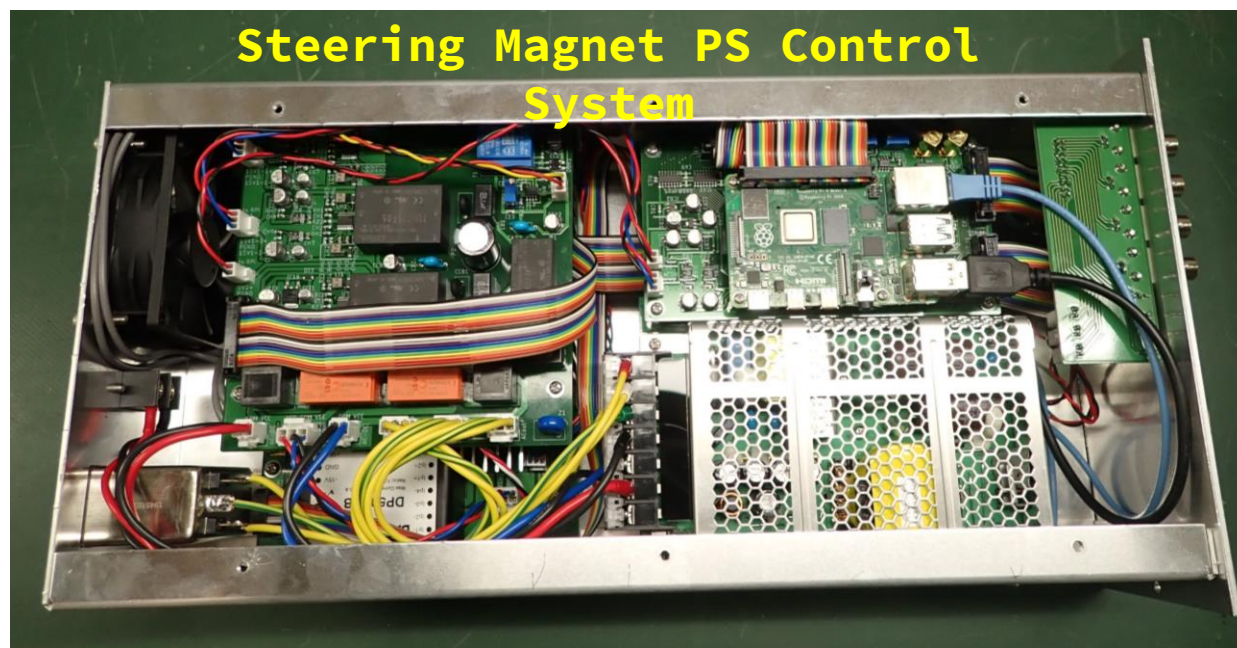
protocols



Example



- ARM board + Adapter board (Arduino) + Sensor
- Arduino programming
- Many analog signals are monitored
- Many PCB boards
- Many cables
- Need to consider noise handling

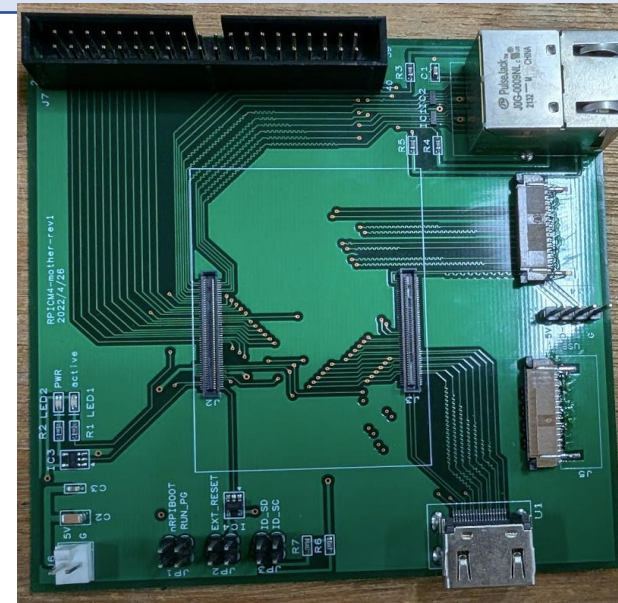
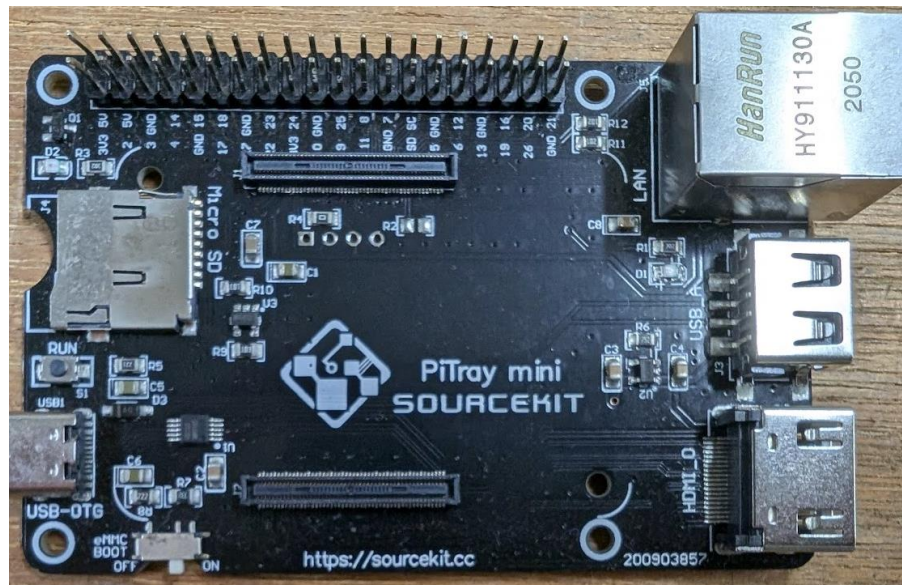
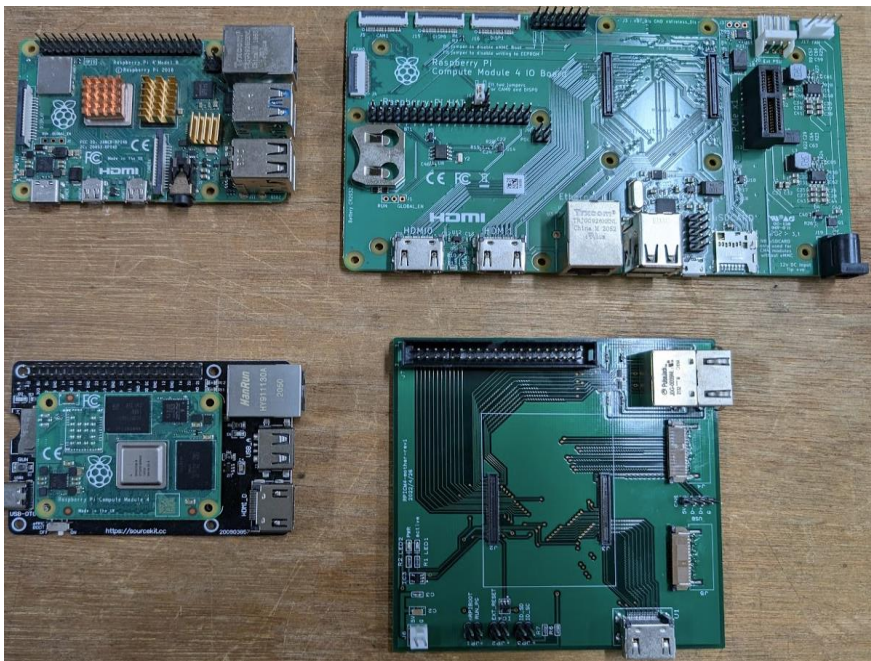


<https://kds.kek.jp/event/35819/>

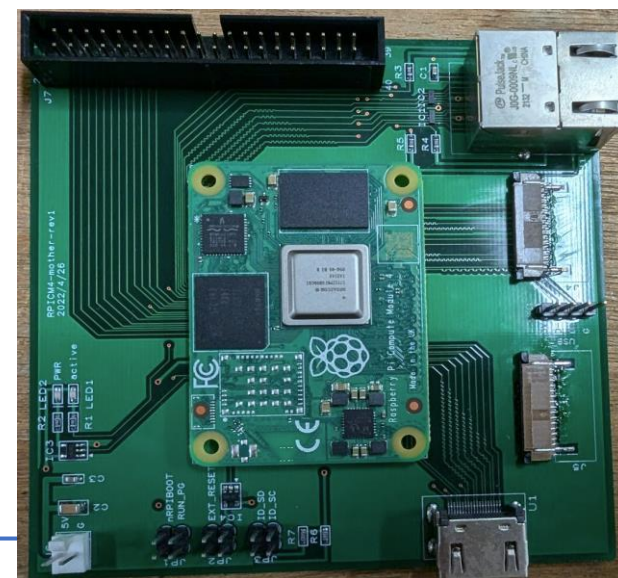
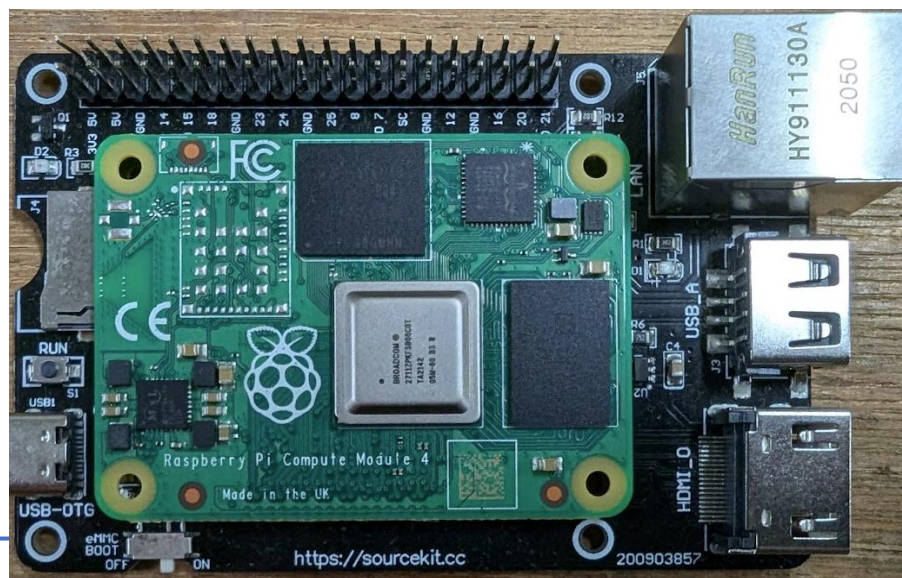
Disadvantages of COTS products

- Commercial off-the-shelf (COTS) ARM board has peripherals which are normal useless for accelerator control
 - Bluetooth, Wi-Fi, Audio Jack, Camera / Display
- These peripherals may even generate annoying noise sometimes
- Depending on the device being controlled/monitored, extra cables / adapter boards are needed
- IO resources are sometimes limited
 - For Raspberry Pi 4B, 28x GPIO are mux-ed with 6x UART, 6x I2C, 5x SPI and others
- System on a Module (**SoM**) may help
 - SoM integrates core components (CPU/FPGA, DDR, flash, power management) on a single PCB
 - The components integrated in the SoM typically requires a high level of interconnection for reasons such as speed, timing, bus width, etc.
 - There are benefits in building a SoM;
 - one notable result is to **reduce the cost** of the base board or the main PCB.
 - Two other major advantages of SoMs are design-reuse and that they can be integrated into many embedded computer applications

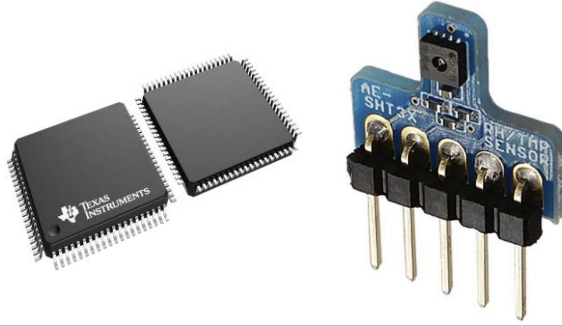
System on a Module (SOM)



- Example usage of a SoM
 - Raspberry Pi Compute Module 4
 - Different carrier/mother board utilize different peripherals



Other possibility?



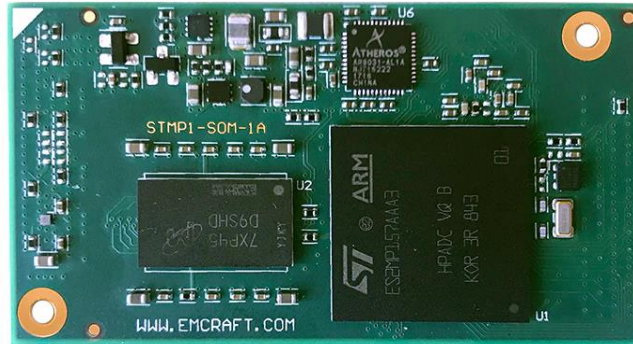
From scratch

- Fully customizable
- Need both good PCB designer and software programmer
- Linux OS porting can be tough

Using SOM



Raspberry Pi Compute Module 4



STM32MP1 System-On-Module

- Simplify the most difficult part of PCB design
 - High speed signal design, like DDR SRAM, is very complex (Need to considerate Signal Integrity and Impedance Matching)
- Several open-hardware and open-source projects can be referred.
- Home-made carrier/mother board
- Rich interface
 - STM32MP157 series have up to 176 GPIOs

COTS



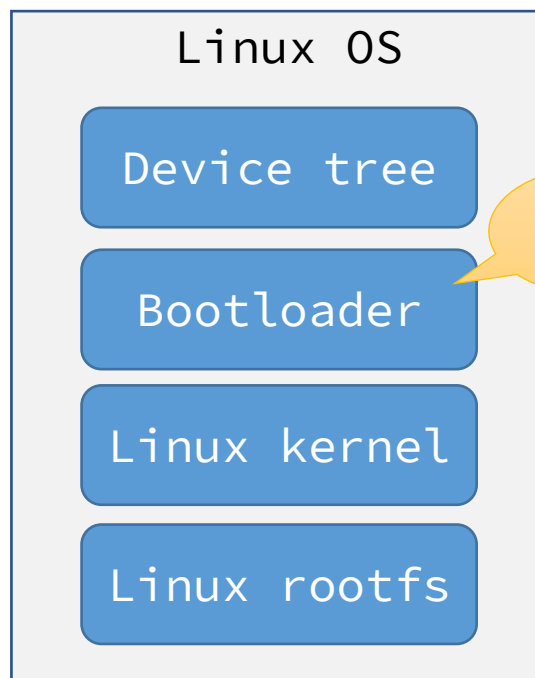
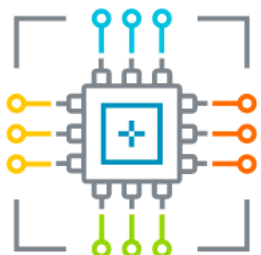
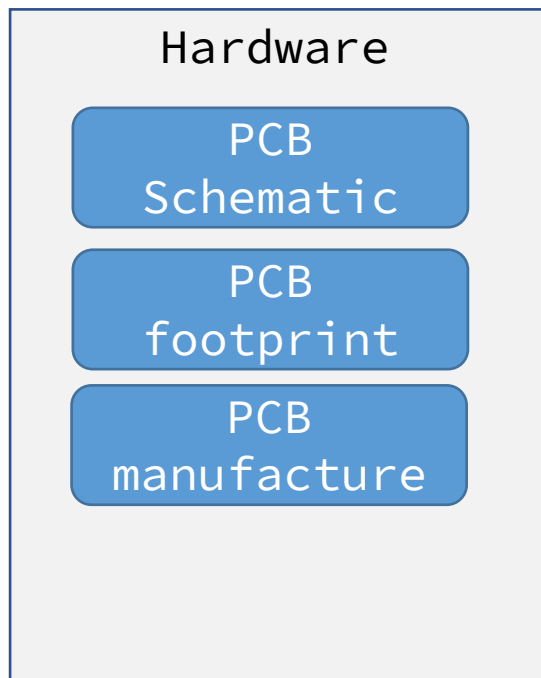
Raspberry Pi 4B



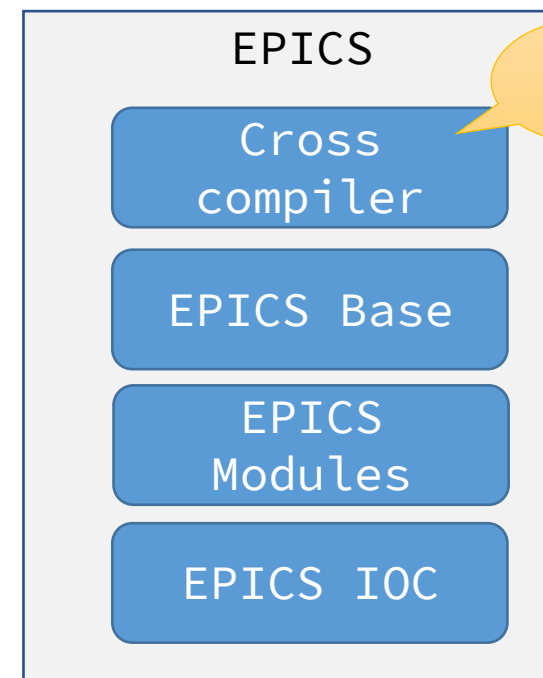
STM32MP157D-DK1

- Hardware connection is simple
- Software is ready to use
- Already used for accelerator for several years
- Extra adapter boards and cables might be needed

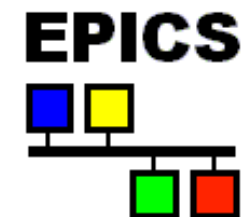
Home-made Embedded control board?



Optional



Optional



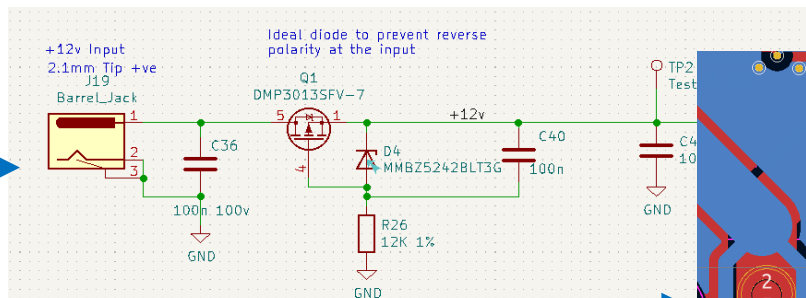
Home-made Embedded control board?

Hardware

PCB
Schematic

PCB
footprint

PCB
manufacture



P板.com | 1-Click見積

アイテム選択



サービス選択 (複数選択可)



基本情報入力

構成層数	片面	2層	4層	6層	8層	10層	12層	14層以上はこちら >
外形寸法	X= 小数第一位まで mm	Y= 小数第一位まで mm						
製造枚数	枚	枚数シミュレーション 量産シミュレーション						

実働	お支払い総額 (税込)	見積明細
-日	-円	
お届け予定日-年-月-日 (-)		
見積もり開始		

1. Select sensors and SOC, finish the connection of electronic circuits

2. Place the component and connect physical wires

3. Use CAD files to order, manufacture and solder

<https://www.p-ban.com>

Home-made Embedded control board?

Linux OS

Device tree

Bootloader

Linux kernel

Linux rootfs

A device tree is [a data structure](#) describing the [hardware components](#) of a particular computer so that the operating system's kernel can use and manage those components, including the CPU, the memory, the buses and the integrated peripherals.

The device tree is then used by bootloader and Linux kernel to setup the board.

4. Usually, the SOC designer provides a sample of the device tree. Only a small modification is required. After compilation of device tree, the board information is used by bootloader and kernel.

```
// SPDX-License-Identifier: GPL-2.0
/dts-v1/;
#include "bcm2711.dtsi"
#include "bcm2835-rpi.dtsi"

#include <dt-bindings/reset/raspberrypi,firmware-reset.h>

/ {
    compatible = "raspberrypi,4-model-b", "brcm,bcm2711";
    model = "Raspberry Pi 4 Model B";

    chosen {
        /* 8250 auxiliary UART instead of pl011 */
        stdout-path = "serial1:115200n8";
    };

    /* Will be filled by the bootloader */
    memory@0 {
        device_type = "memory";
        reg = <0 0 0>;
    };

    aliases {
        emmc2bus = &emmc2bus;
        ethernet0 = &genet;
        pcie0 = &pcie0;
        blconfig = &blconfig;
    };

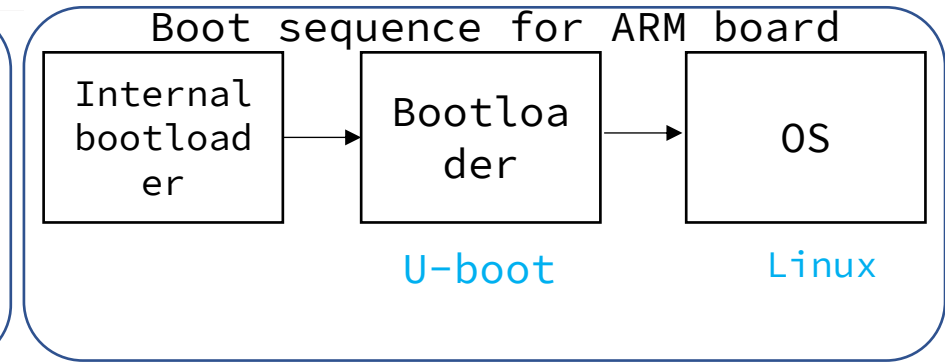
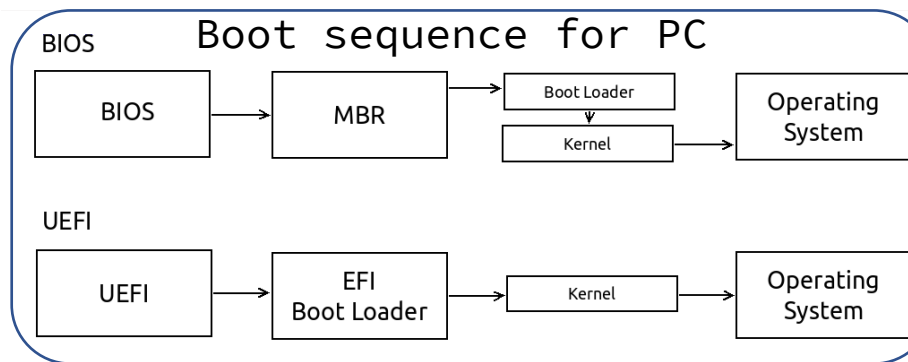
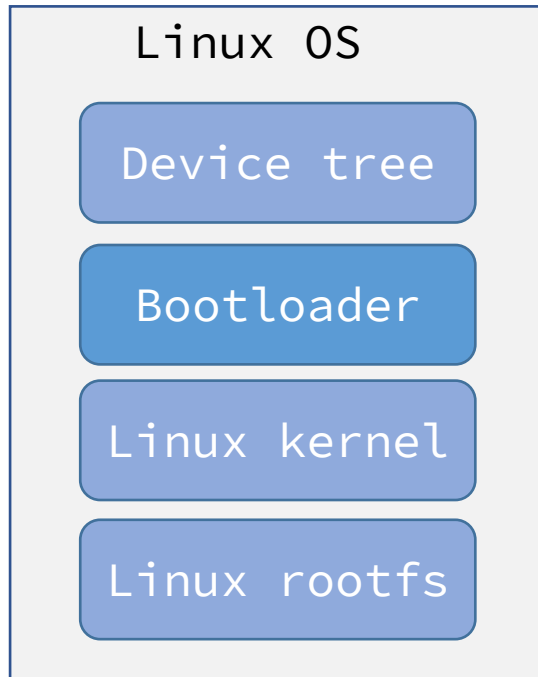
    leds {
        led-act {
            gpios = <&gpio 42 GPIO_ACTIVE_HIGH>;
        };

        led-pwr {
            label = "PWR";
            gpios = <&expgpio 2 GPIO_ACTIVE_LOW>;
            default-state = "keep";
            linux,default-trigger = "default-on";
        };
    };
};
```

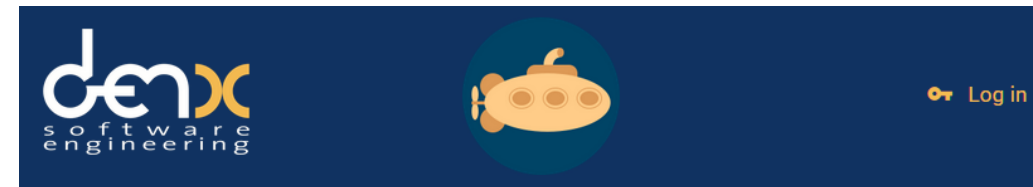
Device tree for RPi 4B

arch/arm/boot/dts/bcm2711-rpi-4-b.dts

Home-made Embedded control board?



5. If the SOC designer already provides the bootloader, this step can be skipped. If not, u-boot can be used to get the bootloader.



You are here: DENX Home » U-Boot

Das U-Boot -- the Universal Boot Loader

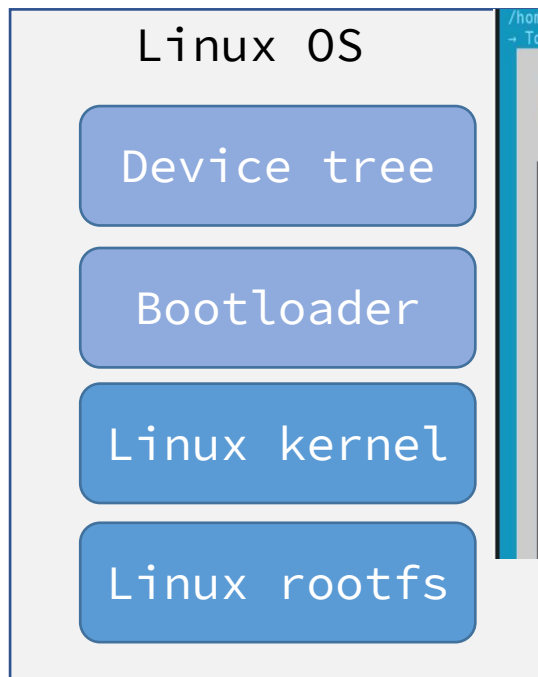
📅 07 July 2022 - 15:45

Welcome to the Wiki area for cooperating on U-Boot development.

💡 **Note:** Documentation on how to use and develop U-Boot can be found at <https://u-boot.readthedocs.io>.

<https://github.com/u-boot/u-boot>

Home-made Embedded control board?



```

/home/sdcswd/workspace/buildroot-2022.02.4/.config - Buildroot 2022.02.4 Configuration
~ Toolchain

Toolchain
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----). Highlighted
letters are hotkeys. Pressing <Y> selects a feature, while <N> excludes a feature. Press <Esc><Esc>
to exit, <?> for Help, </> for Search. Legend: [*] feature is selected [ ] feature is excluded

Toolchain type (Buildroot toolchain) --->
** Toolchain Buildroot Options **
(buildroot) custom toolchain vendor name
[*] C library (glibc) --->
** Kernel Header Options **
Kernel Headers (Same as kernel being built) --->
Custom kernel headers series (5.10.x) --->
** Glibc Options **
[ ] Enable compatibility shims to run on older kernels
[ ] Install glibc utilities
** Binutils Options **
Binutils Version (binutils 2.36.1) --->
Additional binutils options
** GCC Options **
GCC compiler Version (gcc 10.x) --->
Additional gcc options
[*] Enable C++ support
[ ] Enable Fortran support
[ ] Enable D language support

Target Architecture (ARM (little endian)) --->
Target Binary Format (ELF) --->
Target Architecture Variant (cortex-A72) --->
Target ABI (EABIhf) --->
Floating point strategy (NEON/VFPv4) --->
ARM instruction set (ARM) --->

```



Buildroot
Making Embedded Linux Easy

Buildroot is a simple, efficient and easy-to-use tool to generate embedded Linux systems through cross-compilation.

Can handle everything Is very easy Supports several thousand packages

6. Buildroot can generate Linux kernel and Linux rootfs together. Then the OS image is written into the storage(Flash/EMMC/sdCard) of ARM board.

```

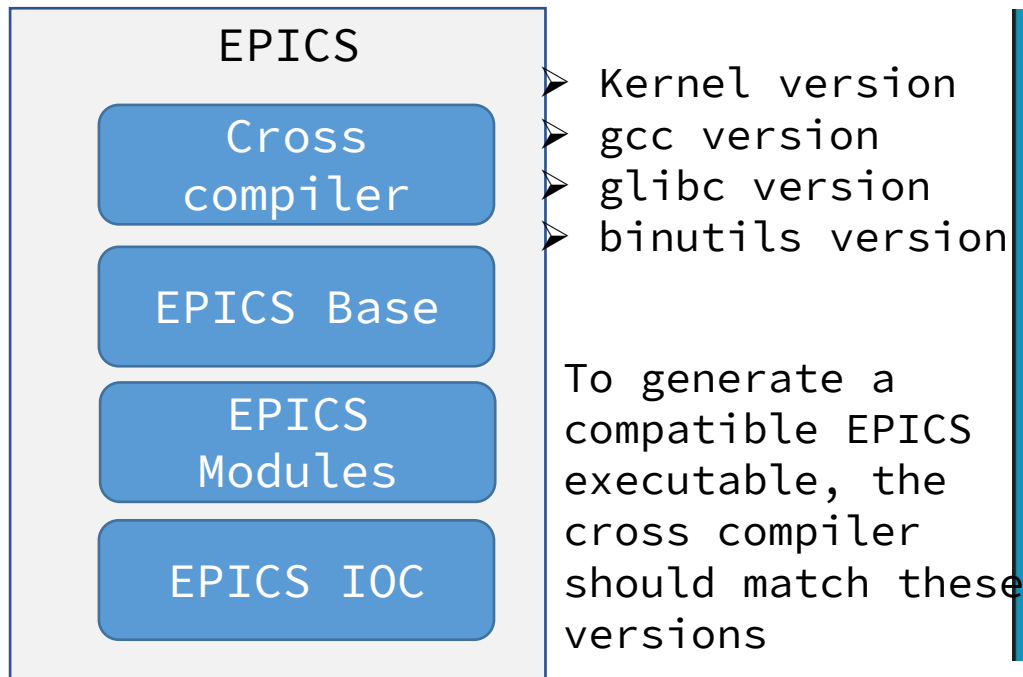
sdcswd @ wang in ~/workspace/buildroot-2022.02.4 [16:44]
$ ls output/images
bcm2711-rpi-cm4.dtb  boot.vfat  rootfs.ext2  rootfs.ext4  rootfs.tar  rpi-firmware  sdcard.img  zImage

```

Device tree Linux rootfs Linux kernel

<https://buildroot.org/>

Home-made Embedded control board?



```
.config - crosstool-NG 1.25.0 Configuration
> C compiler
C compiler
Arrow keys navigate the menu. <Enter> selects submenus ---- (or empty submenus ----).
Highlighted letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes
features. Press <Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] built-in [ ]
excluded <M> module < > module capable

Compiler (gcc) ---->
  *** Options for gcc ***
  Show gcc versions from (GNU) ---->
  Source of gcc (Released tarball) ---->
  [*] Version of gcc (8.5.0) ---->
  ( ) Flags to pass to --enable-cxx-flags
  ( ) Core gcc extra config
  ( ) gcc extra config
  [*] Link libstdc++ statically into the gcc binary
  [ ] Use system zlib
  <M> Configure TLS (Thread Local Storage)
  *** Optimisation features ***
  [*] Enable GRAPHITE loop optimisations
  [*] Enable LTO
```

<https://crosstool-ng.github.io/>

7. Using crosstool-ng to generate cross-compiler toolchain.

Then cross compiling a couple of libraries (libncurses and libreadline) which is needed by EPICS.

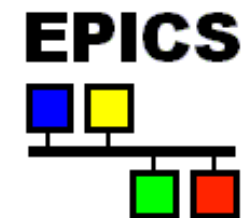
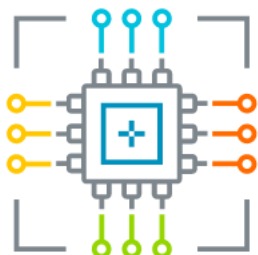
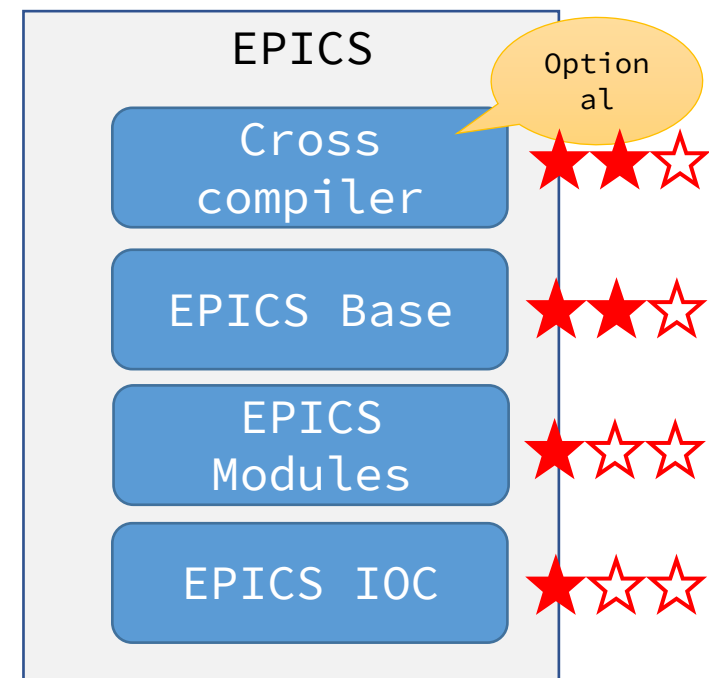
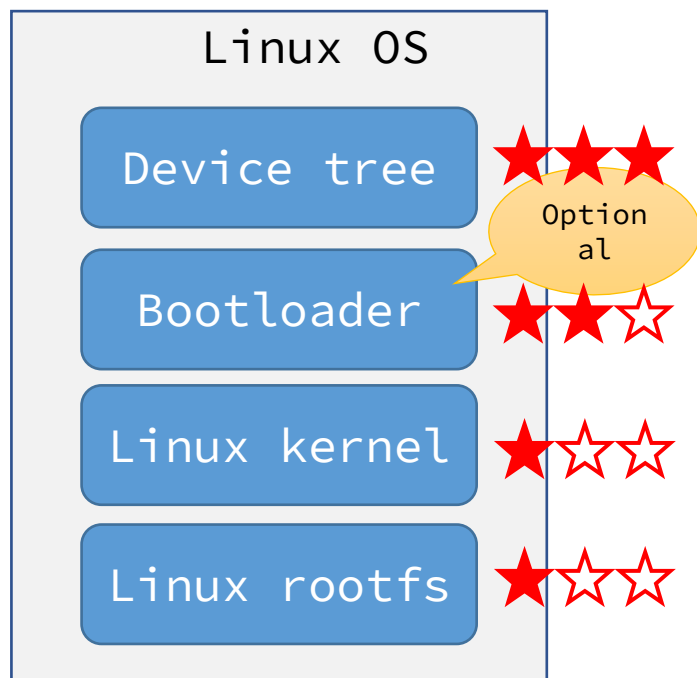
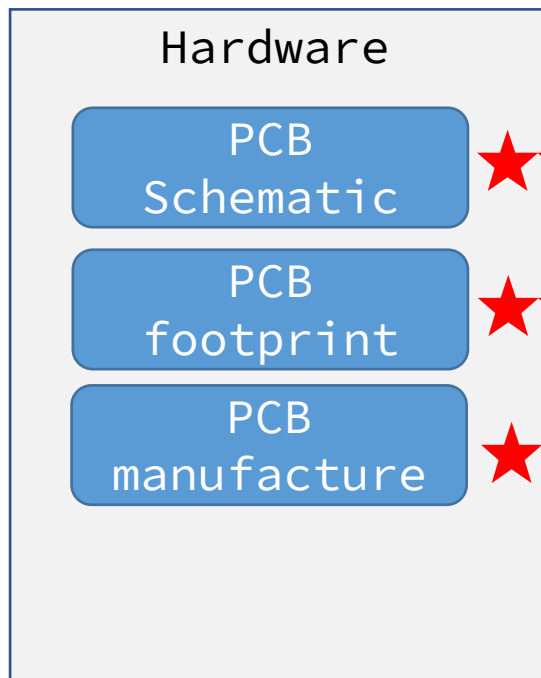
Similarly, generating EPICS modules and IOC.

Select toolchain versions, here gcc 8.5.0 is used

```
sdcswd @ wang in ~/x-tools/arm-rpi4-linux-gnueabi/hf/arm-rpi4-linux-gnueabi/hf/sysroot/usr/lib [17:07]
$ uname -a
Linux wang 5.10.157-1-MANJARO #1 SMP PREEMPT Fri Dec 2 21:02:47 UTC 2022 x86_64 GNU/Linux
sdcswd @ wang in ~/x-tools/arm-rpi4-linux-gnueabi/hf/arm-rpi4-linux-gnueabi/hf/sysroot/usr/lib [17:07]
$ file libreadline.so.6.2
libreadline.so.6.2: ELF 32-bit LSB shared object, ARM, EABI5 version 1 (SYSV), dynamically linked, with debug_info, not stripped
```

ARM-based libreadline is cross-compiled

Difficulties



The future

➤ Citius-Altius-Fortius, Latin for "Faster, Higher, Stronger"

➤ For modular crates

➤ Faster bus speed

➤ PCI Gen 5 support

➤ 10 GbE/40 GbE → 100 GbE

➤ Higher reliability

➤ Redundancy of PM/Cooling/CPU/Switch

➤ Stronger performance

➤ Power per slot

➤ More airflow for cooling

➤ For embedded systems

➤ heterogeneous multiprocessing products

➤ ARM Cortex-A + Cortex-M

➤ ARM Cortex-A for Linux

➤ ARM Cortex-M for real-time applications

➤ NXP i.MX 7/8 series

➤ STM32MP1 series

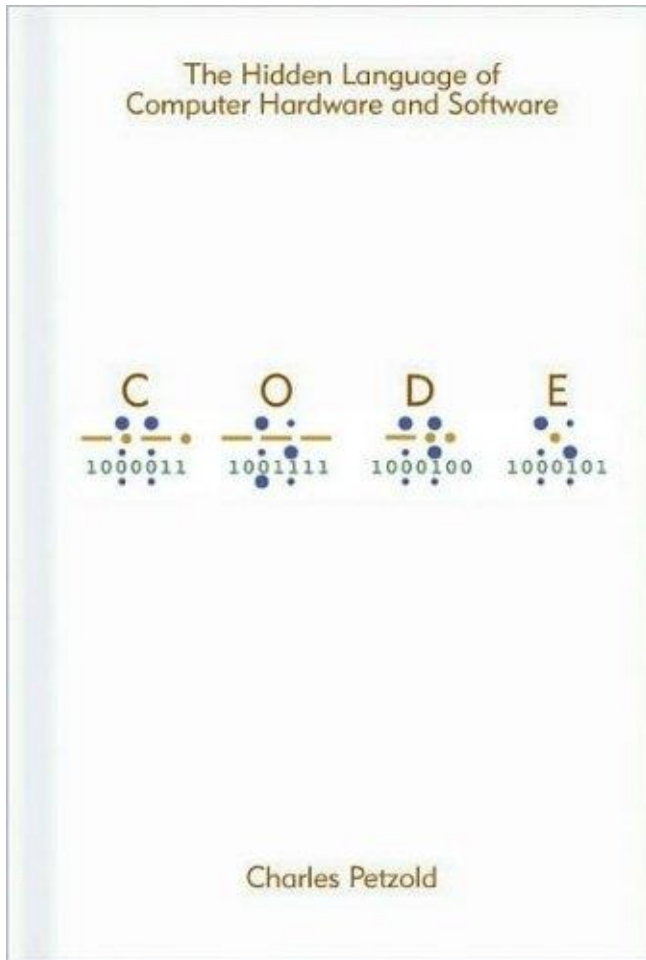
➤ TI AM57x series

➤ ARM Cortex-A + FPGA

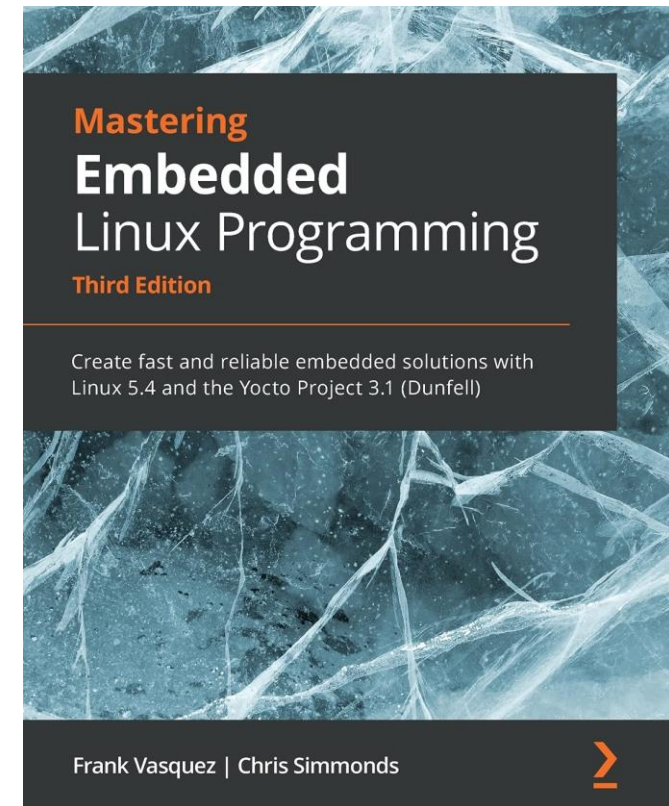
➤ Xilinx Kria K24/K26

Standard	Bandwidth in Mb/s
MicroTCA PCIe Gen5 x4	126 032
MicroTCA PCIe Gen3 x4	31 504
MicroTCA PCIe Gen3 x1	7 877
PXI 64 x 66 MHz	533
VME64x	160
VME64	80
VME16	20

Books/papers worth reading



- Booting Linux: The History and the Future, by Werner Almesberger
- Mastering Embedded Linux Programming, by Chris Simmonds



Code: The Hidden Language of Computer Hardware and Software